

Agroverse Chocolate Subscriptions — pre-flight + execution roadmap

For: TrueSight DAO / Agroverse contributors
From: Claude (agentic AI) — design ratified with Gary Teh
Date: 2026-06-09
Status: Roadmap aligned · implementation begins at Phase 1 PR1.1

Goal: Let supporters subscribe to a recurring monthly shipment of Agroverse ceremonial-cacao **chocolate bars** (generic, vintage-independent), pick the quantity themselves, manage/cancel without operator involvement, and have fulfillment land with **near-zero manual relay** between Gary and Kirsten. First real subscriber: **Linda (Rochester, NY)**. The self-serve subscribe page is also intended as a **cacao-circle event placard QR** target.

► RESUME HERE

► **ACTIVE: Phase 1, PR1.1 — build the** `/subscribe/` **page on** `agroverse_shop_beta`. Nothing implemented yet. This doc was written 2026-06-09 after aligning the design with Gary. Start at the **Pre-flight checklist** (confirm the GAS deploy path + the generic-bar unit price), then PR1.1 in the Execution roadmap table. Each PR is independently shippable; after any PR merges, report the DAO contribution before starting the next (see `DAO_CLIENT_AI_AGENT_CONTRIBUTIONS.md`).

Companion docs (read before touching the relevant layer): - `STRIPE_LEDGER_ROUTING.md` — all 5 Stripe flows; the `[LEDGER_ID]` routing pattern; the Stripe Social Media Checkout ID audit tab. - `notes/claude_serialized_qr_sales_2026-04-29.md` — the one-`[SALES_EVENT]`-per-QR-code pattern + fee amortization (the fulfillment fan-out reuses this verbatim). - `EDGAR_DAO_EXTRACTION_PLAN.md` — Edgar→`dao_protocol` split. **The Stripe webhook ENTRY (`/stripe_webhook`) intentionally stays on Rails (`sentiment_importer`);** new DAO-glue read endpoints go to `dao_protocol (:8010)`. - `DAPP_PAGE_CONVENTIONS.md` + `dapp/UX_CONVENTIONS.md` — for the dapp fulfillment page (Phase 2) and the `agroverse_shop` sign-in (Phase 3).

Design summary (the one insight)

The current one-off flow already records the **physical reality after the fact**: Kirsten picks whatever bars are in stock, then the actual QR codes that shipped are recorded via `report_sales.html`. The clumsiness is the human *relay*, not the model. So:

- **Never pre-assign QR codes.** A subscription is a *recurring charge + a monthly fulfillment obligation*. The specific QR codes are bound at **pack time** by Kirsten, in one batch action. This dissolves the “what if Kirsten can’t find that exact bar” problem — she grabs any N in-stock bars and records what she grabbed.
- **Generic, vintage-independent SKU.** Decouples the subscription commitment from any single harvest. **Traceability is preserved at the unit level** — every bar still carries its QR → vintage → farm; the subscriber *discovers* provenance by scanning (the Cacao Chasers surprise model), they just don’t pre-select.

- **Fulfillment = N existing [SALES EVENT] s**, not a new event type. The entire QR→ledger→treasury chain runs unchanged; revenue auto-routes to each bar's own AGL ledger via its `ledger_shortcut`. Only genuinely new pieces: the obligation **queue** and a **batch-submit UI** that loops (≈ `dao_client/examples/bulk_qr_sales_template.py`).
- **Identity = verified email, server is source of truth**. Reuse the canonical RSA flow (capoeira/oracle/dapp). Cross-browser portability is solved by making the verified email the key and reading orders/subscriptions **server-side** — `localStorage (agroverse_order_history)` becomes a cache, never synced between browsers. Subscription management = a **Stripe Customer Portal** link minted for the matching Customer (works from any browser).

Decisions (ratified with Gary, 2026-06-09)

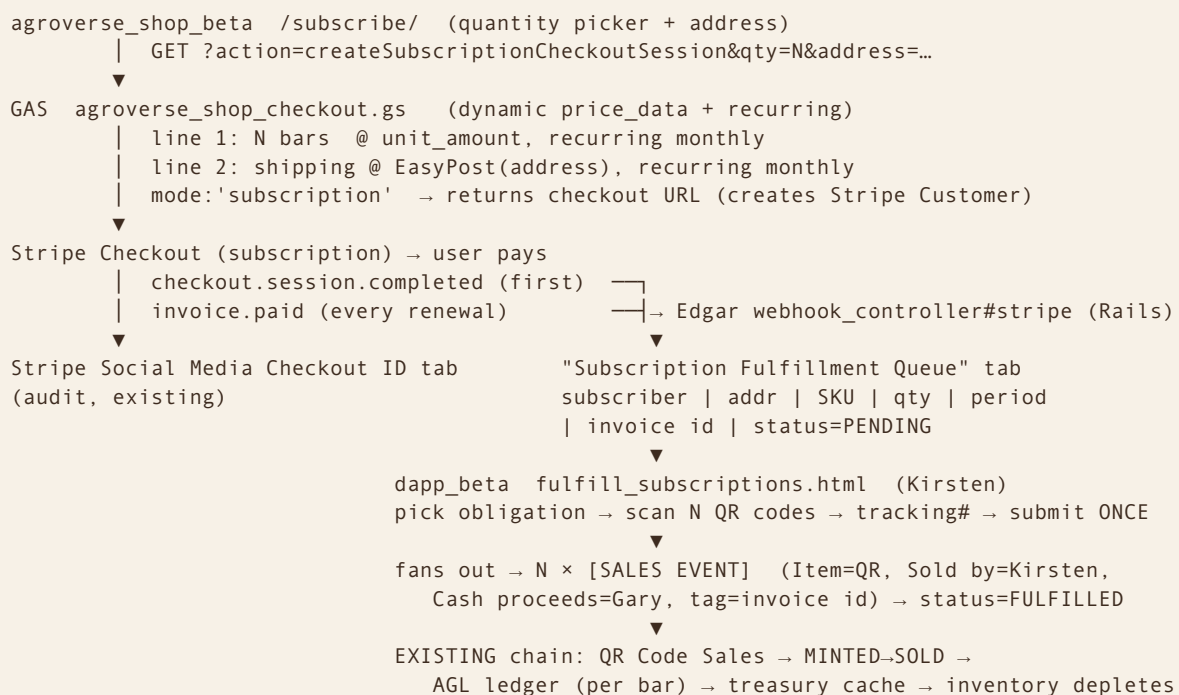
#	Decision
Scope	Chocolate bars only for now (no generic ceremonial-cacao SKU yet).
Billing engine	Stays in the existing GAS (<code>agroverse_shop_checkout.gs</code> → <code>createCheckoutSession</code>). Subscriptions reuse the same dynamic inline price_data , adding <code>recurring:{interval:'month'} + mode:'subscription'</code> . No pre-created Stripe Price IDs. Confirmed: Stripe Checkout supports on-the-fly recurring <code>price_data</code> .
Quantity	User-chosen bars/month on the subscribe page (also the event-placard QR target).
Pricing	Same basis as the shop today — generic bar unit price × quantity (dynamic), computed server-side.
Shipping	Computed from signup address — run EasyPost once at signup, lock that amount as a recurring shipping line item (subscription mode has no interactive <code>shipping_options</code> picker, so a recurring line is the correct mechanism).
Fulfillment attribution	On each per-bar [SALES EVENT] : Sold by = Kirsten Ritschel, Cash proceeds collected by = Gary Teh (same as today).
Start order	Phase 1 (the subscribe page) first ; Linda receives that page's prod URL (no throwaway zero-code Stripe link).
Identity	Adopt the canonical RSA email-bound flow on <code>agroverse_shop</code> (Phase 3); server-side read-by-email retires the <code>localStorage</code> -only history limitation.
Beta-first	All UI work lands in <code>agroverse_shop_beta</code> / <code>dapp_beta</code> ; human promotes to prod. Never push prod directly.

Pre-flight checklist (verify BEFORE writing code)

- [] **GAS deploy path for the checkout script**. Confirm which clasp mirror deploys `agroverse_shop/google-app-script/agroverse_shop_checkout/agroverse_shop_checkout.gs` (the in-repo copy is reference; deploy from `tokenomics/clasp_mirrors/<scriptId>/` per `NOTES_tokenomics.md`). Capture the `scriptId` + the existing `/exec` deployment URL so subscription sessions deploy without breaking the cart.
- [] **Generic-bar unit price**. Confirm the canonical retail bar price from `agroverse_shop/js/products.js` (Elizabeth Wong batch was \$10.00/bar — verify it's still current) and the exact **generic SKU name** for `product_data.name` (proposed: `Agroverse Ceremonial Cacao Chocolate Bar — Monthly`).
- [] **Stripe test mode**. Confirm the `sk_test` sandbox path (used for the PR6a QR-checkout work) is available to validate `mode:'subscription'` end to end before any prod promotion.

- [] **Webhook coverage.** Confirm `sentiment_importer/app/controllers/webhook_controller.rb#stripe` currently handles only `checkout.session.completed`; Phase 2 adds `invoice.paid` (recurring renewals). Note: the **first** charge arrives as `checkout.session.completed` (mode=subscription); **renewals** arrive as `invoice.paid` — both must create a fulfillment obligation.
- [] **Downstream session-id prefix.** The sales parser validates `cs_(live|test)_` prefixes and strips `(none)`. Subscription renewals carry `in_.../sub_...`, not `cs_...`. Decide: teach the parser the new prefix, or stash the invoice id in a separate attr and leave “Stripe Session ID” blank. (Phase 2, PR2.4.)
- [] **Sheet writes.** Confirm the service account that may write the new “Subscription Fulfillment Queue” tab (likely `agroverse-qr-code-manager@...`); `Currencies` stays range-protected (not touched here).
- [] **dao_client auth active** for contribution reporting (`auth.py status`).

Architecture (where each piece lives)



Execution roadmap (resume tracker)

Legend: ☐ todo · ☒ in progress · ☒ merged · 🗳️ DAO contribution reported

Phase 1 — Self-serve subscribe page (START HERE)

PR	Repo	Scope	Status
1.1	agroverse_shop_beta	<code>/subscribe/</code> page: bars/month quantity picker + address form (reuse <code>checkout-form-storage.js</code>), calls the new GAS action, redirects to Stripe. Follow <code>DAPP_PAGE_CONVENTIONS</code> equivalents for the shop.	☐

PR	Repo	Scope	Status
1.2	GAS (checkout mirror)	New <code>createSubscriptionCheckoutSession</code> action: bars line item with <code>recurring:{interval:'month'}</code> ; run existing EasyPost weight+rate code once for the address → recurring shipping line item; <code>mode:'subscription'</code> ; return <code>checkoutUrl</code> . Reuse the cart action's price/weight helpers.	☐
1.3	<code>agroverse_shop_beta</code>	Define the generic bar SKU (name, unit price, image) for <code>product_data</code> ; document it next to <code>products.js</code> .	☐
1.4	GAS / verify	Confirm subscription-mode <code>checkout.session.completed</code> still logs to the Stripe Social Media Checkout ID tab (audit) via the existing webhook path; patch if subscription sessions differ.	☐
1.5	promote + handoff	Beta → prod promotion (human), then send Linda the prod <code>/subscribe/</code> URL ; smoke-test one real subscription in test mode first.	☐

Phase 2 — Fulfillment automation (kill the relay)

PR	Repo	Scope	Status
2.1	Google Sheets	Create “ Subscription Fulfillment Queue ” tab + schema (subscriber, email, address, SKU, qty, period, invoice id, status, fulfilled-by, tracking#, fulfilled-at).	☐
2.2	<code>sentiment_importer</code> (Rails)	Handle <code>invoice.paid</code> and subscription-mode <code>checkout.session.completed</code> in <code>webhook_controller#stripe</code> → append a PENDING obligation row (new <code>Gdrive::*</code> writer). Idempotent on invoice id.	☐
2.3	<code>dapp_beta</code>	<code>fulfill_subscriptions.html</code> : list PENDING obligations, pick one, scan/enter N QR codes (reuse <code>report_sales.html</code> + <code>edgar_payload_helper.js</code>), enter tracking#, submit once → loop N <code>[SALES EVENT]</code> s (Sold by=Kirsten, Cash proceeds=Gary, tag invoice id) → mark FULFILLED.	☐
2.4	GAS (sales parser)	Accept subscription invoice/ <code>sub_ids</code> (or read a dedicated attr) so renewal-sourced <code>[SALES EVENT]</code> s reconcile without the <code>cs_</code> guard tripping.	☐
2.5	<code>dapp_beta</code> (stretch)	Low-stock alert on the queue: warn when generic-pool stock < upcoming obligations (supply-buffer second-order effect).	☐

Phase 3 — RSA accounts + cross-browser portability

PR	Repo	Scope	Status
3.1	<code>agroverse_shop_beta</code>	“Sign in” in nav: reuse canonical RSA flow (<code>publicKey/privateKey</code> localStorage, <code>edgar_payload_helper.js</code> , <code>[EMAIL REGISTERED EVENT]</code> →link→ <code>[EMAIL VERIFICATION EVENT]</code>).	☐
3.2	Google Sheets / Rails	Add buyer email column to the Stripe Social Media Checkout ID tab (write it on <code>append_record</code>); backfill from Stripe where feasible.	☐
3.3	<code>dao_protocol</code>	Read-by-verified-email endpoint → returns the email's orders (Stripe tab) + active subscriptions (Stripe API).	☐
3.4	<code>dao_protocol</code>	Mint a Stripe Customer Portal session for the verified email's Customer; “Manage subscription” button on the shop.	☐

PR	Repo	Scope	Status
3.5	agroverse_shop_beta	/order-history/ reads the server endpoint when signed in (localStorage = fallback cache); one-time migration of local agroverse_order_history on first verify.	☐

Risks / open items

- **Supply buffer.** Subscriptions create a recurring inventory commitment; the generic pool must be replenished as vintages finish. PR2.5 surfaces it; supply planning is out of scope for code but in scope for Dr-Manhattan-level attention.
- **Shipping drift.** Locked-at-signup shipping can diverge from real cost over time (carrier increases, address-region changes). Acceptable for v1; revisit if margins slip. Customer Portal lets subscribers update address — decide whether an address change should re-quote shipping (likely a manual re-quote at first).
- **Deferred revenue timing.** Charge lands on billing day; revenue recognized when QRs are scanned days later. The obligation queue is the deferred-revenue tracker — correct, but means the Stripe tab and per-bar [SALES EVENT] s are temporarily out of sync within a cycle (expected).
- **Single-vs-multi service shipping.** EasyPost returns multiple USPS rates; v1 picks one tier (e.g. Ground Advantage) computed for the address rather than letting the subscriber choose. Confirm the tier in PR1.2.

Plan owner: this doc. Update the resume tracker as each PR lands; report the DAO contribution before starting the next PR.